Assignment 1 Report

Aadil Goyal

October 21, 2026

Abstract

The GitHub Repo can be found at link and the STM Board used is stm32f103c8t6

1 Question 1

Question: Cats or Dogs?

Ans: Not really into either, but if I had to choose then Dogs

2 Question 2

Question: What is the difference between UART and USART? Would you prefer working with UART or USART while working on the rover (imagine you want to communicate between the Raspberry Pi and an STM32 bluepill to transfer motor commands and telemetry data)? Explain why.

Ans: UART stands for Universal Asynchronous Receiver Transmitter while USART stands for Universal Synchronous Asynchronous Receiver Transmitter
More differences are as follows.

Feature	UART	USART
Full Form	Universal Asynchronous Receiver Transmitter	Universal Synchronous/Asynchronous Receiver Transmitter
Communication Type	Asynchronous only	Supports both Synchronous and Asynchronous
Clock Signal	Not used	Uses an additional clock line in synchronous mode
Number of Pins Used	2 (TX and RX)	3 in synchronous mode (TX, RX, and CLK)
Data Sampling	Based on start and stop bits (internal clock)	Data is sampled on the rising or falling edge of external clock
Application Example	Basic serial communication with PCs, Bluetooth modules	Smartcard communication, SPI-like data transfers
Acts Like SPI Master?	No	Yes, in synchronous mode
Use of Start/Stop Bits	Required for data framing	Optional in synchronous mode

I would prefer working with USART because it gives higher speeds and better reliability. This is extremely important for transferring motor commands and telemetry data inside the rover.

3 Question 3

Question: If I set the TIMEOUT parameter of HAL UART Receive to 0, what would happen? Hint: Would the code still be blocking?

Ans: If we set TIMEOUT to 0, then it will do the receiving of signal for 0 milliseconds only. It will be non blocking (as it blocks for 0ms), but as it is only receiving for 0ms, it wont be able to do the receiving either. So it would defeat the purpose of calling the function in the first place.

4 Question 4

Question: Explain all the code you have written briefly.

Ans: Inside the while loop, we first try to receive a signal for 5 secs

If we receive a signal (*uart_response* is *HAL_OK*), we call the function *ExecuteCommand* which checks whether the received signal is 0 or 1 or something else. On the basis of this, it eithers sets the inbuilt led to OFF or ON or sends "Invalid Data!"

Or If we get no reponse (*uart_response* is *HAL_TIMEOUT*), we transmit the "I'm feeling lonely :(" msg. Then we check for another 2 seconds

If there is still no signal, we transmit another lonely msg

If there is a signal we call the *ExecuteCommand* function

This code uses polling as we are calling the *HAL_UART_Receive* function and waiting for a signnal until *TIMEOUT* and hence it is also blocking

5 Question 5

Question: Explain how interrupts work for transmission and how it is different from interrupts for receiving.

Ans: In an Tx Interrupt, we first send an interrupt signal. Then when we get a response, then we send the data that needs to be sent. In an Rx interrupt, we get an interrupt signal, then we check its priority. Then the CPU suspends its current task and saves its state. Then the CPU jumps to the interrupt handler (ISR) and handles the device's request. After completing the request, the interrupt handler restores the CPU's previous state and it resumes its task.

6 Question 6

Question: Explain the logic you have used for implementing the task using interrupts briefly.

Ans: We first call *HAL_UART_Receive_IT*, so as soon as we get a signal, we go the the Rx Handler (*HAL_UART_RxCpltCallback*) and check whether the msg is 0 or 1 or something else. Depending on it we set the inbuilt led to OFF or ON or transmit "Invalid Data". Finally we update the *last_time* to the current time and *isLonely* to *false*

Parallely in the while loop, we first check *isLonely*. If *isLonely* is *false* (we just received a signal), so we check whether it has been 5 secs since that happened. If it did then we transmit the lonely msg and switch *isLonely* to *true* and *last_time* to rn. If *isLonely* is *true* (we are now in the 2 sec stage), so we check whether it has been 2 secs. If it has been then we transmit a lonely msg and update *last_time*.

7 Question 7

Question: Explain how DMA works for transmission and how it is different from DMA for receiving.

Ans: When a peripheral device needs to transfer data to or from memory, it sends a DMA request signal to the DMA controller. This signal indicates that the CPU is not required for this particular data transfer operation.
Upon receiving the DMA request, the DMA controller checks if the bus is available and then initiates access to memory. By handling these requests independently of the CPU, DMA significantly reduces processor overhead and speeds up data transfers between devices and memory.
The DMA controller manages the timing and prioritization of these requests through efficient arbitration techniques. This ensures that multiple devices can communicate with memory seamlessly without causing conflicts or bottlenecks in data flow.

8 Question 8

Question: Explain the logic you have used for implementing the task using DMA briefly.

Ans: For this I have used the exact same code as for interrupt, only changed IT to DMA